

# LangSmith 소개

---

대규모 언어 모델(LLM) 기반 애플리케이션 개발을 위한 올인원 플랫폼



추적 & 디버깅



테스트 & 평가



모니터링 & 분석

# LangSmith란 무엇인가

## 정의

LangSmith는 LangChain에서 개발한 LLM(대규모 언어 모델) 애플리케이션 개발 및 운영을 위한 **올인원 플랫폼**입니다.

## 목표

LLM 기반 애플리케이션의 프로토타이핑 단계부터 프로덕션 단계까지의 전 과정을 지원합니다.



개발 가속화



성능 최적화



신뢰성 확보



협업 강화

## 핵심 특징

- ✓ LLM 애플리케이션 특화
- ✓ 전체 개발 주기 지원
- ✓ 디버깅 및 추적 도구
- ✓ 테스트 및 평가 시스템
- ✓ 모니터링 및 분석 대시보드

🔗 프로토타이핑에서 프로덕션까지

# 핵심 기능



## 추적 및 디버깅

LLM 애플리케이션의 모든 실행 과정을 시각적으로 추적하고 문제를 신속히 디버깅할 수 있습니다.

- ✓ 입출력, 지연 시간, 토큰 사용량 상세 확인
- ✓ 체인(Chain)이나 에이전트(Agent) 실행 흐름 그래프화
- ✓ 문제 발생 단계 및 원인 신속 파악

실행 흐름 시각화



## 테스트 및 평가

LLM 애플리케이션의 성능을 정교화하여 체계적으로 평가할 수 있는 테스트 기능을 제공합니다.

- ✓ 정확성
- ✓ 일관성
- ✓ 지연 시간
- ✓ 관련성
- ✓ 유해성/편향성
- ✓ 토큰 사용량

정량적 평가 지표



## 모니터링 및 분석

프로덕션 환경에서 LLM 애플리케이션의 성능과 비용을 실시간으로 모니터링합니다.

- ✓ 응답 시간, 오류율, 처리량 등 성능 지표
- ✓ LLM API 호출 비용, 토큰 사용량 분석
- ✓ 사용자 피드백 및 모델 버전별 성능 비교

실시간 대시보드

LangSmith는 이러한 핵심 기능들을 통합하여 LLM 애플리케이션 개발의 전 과정을 지원합니다.

# 기대 효과



## 개발 생산성 향상

복잡한 디버깅 및 테스트 과정을 자동화하고 단순화  
핵심 로직 개발에 집중할 수 있도록 지원  
전체 개발 사이클을 단축시켜 빠른 애플리케이션 출시 가능  
수동 오류 찾고 수정에 소요되는 시간을 줄임

✓ 개발 효율성 극대화



## 운영 신뢰성 확보

실시간 모니터링과 피드백 루프 통해 예기치 않은 오류 신속하게 대응  
서비스의 안정적인 운영을 보장하여 사용자 경험 저해 최소화  
성능 저하를 조기에 감지하고 잠재적 문제 예측  
API 호출 지연이나 응답 품질 저하를 신속히 식별하고 해결

✓ 서비스 안정성 향상

➔ LangSmith를 도입하면 LLM 애플리케이션의 품질과 신뢰성을 동시에 향상시킬 수 있습니다

# 결론

LangSmith는 LLM(대규모 언어 모델) 기반 애플리케이션의 **개발부터 운영까지 전 과정의 품질과 신뢰성을 보장하는 필수적인 플랫폼으로 자리매김하고 있습니다.**



## 개발 생산성 향상

복잡한 디버깅 및 테스트 과정을 자동화하고 단순화하여 핵심 로직 개발에 집중할 수 있도록 지원합니다.



## 운영 신뢰성 확보

실시간 모니터링과 피드백 루프를 통해 예기치 않은 오류나 성능 저하에 신속하게 대응할 수 있습니다.



## 지속적인 개선 가능

모니터링 데이터와 사용자 피드백을 기반으로 애플리케이션을 지속적으로 개선하고 업데이트할 수 있습니다.

LangSmith와 함께라면 LLM 기반 애플리케이션의 성공적인 배포와 지속적 개선이 가능합니다.

# Ragas와 평가 지표

RAG(검색 증강 생성) 시스템의 효율적 평가를 위한 종합적 프레임워크



평가 지표



검색 평가



LLM 평가

# RAG 시스템과 평가의 필요성



## RAG 시스템 개요

- ✓ 대규모 언어 모델(LLM)의 한계를 극복하기 위해 개발됨
- ✓ 최신 정보와 특정 도메인 지식을 활용하여 답변 생성



## 기존 평가 방식의 한계

- ✗ 정답(Ground Truth) 데이터셋에 의존함
- ✗ RAG 시스템의 복잡성으로 인해 모든 구성 요소를 효과적으로 평가하기 어려움



## Ragas 도입 배경

### 혁신적인 접근 방식

Ragas는 Ground Truth 데이터 없이도 RAG 파이프라인의 각 구성 요소를 독립적으로 평가할 수 있는 새로운 접근 방식을 제공합니다.

### 💡 Ragas의 “Self-evaluation” 기법이란?

LLM을 평가자(Judge)로 활용해 정답 데이터가 없이도 RAG의 성능을 평가하는 기법입니다.

LLM이 문맥과 모델이 생성한 답변을 보고 LLM 자체가 판단 기준을 만들어 평가합니다.

# Ragas의 핵심 개념

Ragas는 Ground Truth(정답) 데이터에 의존하지 않고도 RAG 시스템의 성능을 객관적으로 평가할 수 있는 새로운 접근 방식을 제시합니다. 이는 RAG 파이프라인의 각 구성 요소를 독립적으로 분석함으로써 가능합니다.

## 💡 평가 철학



### 독립적인 구성 요소 평가

검색된 문맥의 관련성, 완전성, 생성된 답변의 충실도, 관련성 등을 개별적으로 측정합니다.



### LLM 기반 평가

LLM을 활용하여 사용자 질의와 생성된 답변 간의 유사성을 평가합니다. 정답 데이터 없이도 의미적인 평가를 가능하게 합니다.



### 데이터 의존성 감소

정답 데이터셋 구축에 드는 시간과 비용을 줄여, 개발 초기 단계부터 효율적인 평가를 가능합니다.

## ⚙️ RAG 파이프라인의 독립적 평가 방식



★ 효과: RAG 시스템의 특정 구성 요소에서 발생하는 문제를 정확히 진단하고, 시스템 전반의 성능을 체계적으로 개선할 수 있는 기반을 제공



# RAG 파이프라인 구조



## 검색기 (Retriever)

- ✓ 방대한 외부 지식 기반에서 질의와 관련된 정보를 찾아냄
- ✓ 검색기의 성능은 최종 답변의 품질에 결정적인 영향을 미침
- ✓ 관련성이 낮은 정보가 검색되거나 중요한 정보가 누락되면 생성기의 답변 품질 저하

## 생성기 (Generator)

- ✓ LLM 기반의 생성기는 검색된 정보를 바탕으로 답변 생성
- ✓ 정보를 종합하고 재구성하여 일관성 있고 유용한 답변 생성
- ✓ 생성기의 품질은 답변의 유창성, 일관성, 관련성에 영향을 미침

# 검색(Retriever) 관련 평가 지표

RAG(Retrieval-Augmented Generation) 시스템의 성능을 평가하기 위해 Ragas는 다양한 메트릭을 제공합니다. 메트릭들은 LLM(Large Language Model)을 평가자로 활용하여 생성된 답변의 품질과 검색된 컨텍스트의 관련성을 다각도로 측정합니다.



## Context Precision

검색된 문서가 질문과 관련 있는가?

- 검색된 컨텍스트의 질을 측정
- 정답에 필요한 정보만을 포함하도록 유도



## LLM Self-evaluation

RAGAS는 내부적으로 "평가자 LLM"에게 질문합니다:

Given the **question and each context chunk**,  
does this context chunk support answering the  
question?

Answer yes or no.

LLM이 'Yes' -> Verdict = 1 (필요한 정보 포함)

LLM이 'No' -> Verdict = 0 (관련 없는 문서)

# 검색(Retriever) 관련 평가 지표

RAG(Retrieval-Augmented Generation) 시스템의 성능을 평가하기 위해 Ragas는 다양한 메트릭을 제공합니다. 메트릭들은 LLM(Large Language Model)을 평가자로 활용하여 생성된 답변의 품질과 검색된 컨텍스트의 관련성을 다각도로 측정합니다.

## Context Recall

정답에 필요한 정보가 검색된 Context 문서에 모두 포함되어 있는가?

- 답변에 필요한 모든 정보를 포함하고 있는지 평가  
검색된 컨텍스트가 충분한지 측정 (정보 누락 여부)



## LLM Self-evaluation

RAGAS는 내부적으로 "평가자 LLM"에게 질문합니다:

- Given the **ground truth and the retrieved context**, is all information needed to answer the ground truth present in the context?  
Answer yes or no.
- LLM이 'Yes' -> Verdict = 1 (필요한 정보 포함)  
LLM이 'No' -> Verdict = 0 (관련 없는 문서)

# 생성(Generator) 관련 평가 지표

RAG(Retrieval-Augmented Generation) 시스템의 성능을 평가하기 위해 Ragas는 다양한 메트릭을 제공합니다. 메트릭들은 LLM(Large Language Model)을 평가자로 활용하여 생성된 답변의 품질과 검색된 컨텍스트의 관련성을 다각도로 측정합니다.



## Faithfulness (충실도)

모델의 답변이 Retrieval 문서(Context)에 기반하여 사실 그대로 반영하고 있는가?

- 환각(factual hallucinations)을 방지하는 데 중점
- 생성된 답변의 각 주장이 컨텍스트에서 추론 가능한지 검증



## LLM Self-evaluation

RAGAS는 내부적으로 "평가자 LLM"에게 질문합니다:

- Given the **context and the answer**,  
is every part of the answer fully supported by the context?  
Answer yes or no.
- LLM이 'Yes' -> Verdict = 1  
LLM이 'No' -> Verdict = 0 (환각이 있는 경우)

# 생성(Generator) 관련 평가 지표

RAG(Retrieval-Augmented Generation) 시스템의 성능을 평가하기 위해 Ragas는 다양한 메트릭을 제공합니다. 메트릭들은 LLM(Large Language Model)을 평가자로 활용하여 생성된 답변의 품질과 검색된 컨텍스트의 관련성을 다각도로 측정합니다.



## Answer Relevancy (답변 관련성)

모델이 생성한 답변이 질문과 얼마나 관련성이 있는가?

- 답변이 질문에 직접적으로 관련 있는지 측정
- 의도된 정보 전달의 효과를 평가  
(답변이 질문의 의도에 부합했는가?)



## LLM Self-evaluation

RAGAS는 내부적으로 "평가자 LLM"에게 질문합니다:

Given the **question and the model answer**,  
is the answer relevant and helpful for answering the  
question?

Answer yes or no.

LLM이 'Yes' -> Verdict = 1 (질문과 관련 있음)

LLM이 'No' -> Verdict = 0 (질문과 무관)

# Ragas의 가치와 전망



## Ragas의 핵심 가치

- ✓ RAG 시스템의 핵심 구성 요소인 검색과 생성을 독립적이고 종합적으로 평가
- ✓ Ground Truth 데이터 없이도 효과적인 평가 가능



## 향후 전망

- ➔ 다양한 평가 지표 통합
- ➔ 자동화된 분석 기능 확장
- ➔ LLM 기반 시스템의 품질 보증 및 최적화에 있어 중요 역할 수행



## LLM 애플리케이션 품질 보증 역할



검색 성능 분석



생성 품질 평가

“

Ragas는 신뢰할 수 있는 LLM 애플리케이션 구축을 위한 필수적인 도구로 자리매김하며, 지속적인 발전을 통해 LLM 기반 시스템의 품질과 성능을 높일 것입니다.

# LangSmith와 Ragas를 활용한 오프라인 평가 구성

합성 테스트 데이터 생성, 저장, 평가 전과정



합성 테스트 데이터 생성

Ragas를 통한 자동화



LangSmith Dataset

저장

관리 및 공유



Ragas 평가 매트릭 적용

체계적 분석

# 오프라인 평가의 개요 및 필요성

## LLM 기반 RAG 시스템 평가

LangSmith와 Ragas를 활용한 오프라인 평가는 LLM 기반 애플리케이션, 특히 RAG 시스템의 성능을 체계적으로 평가하고 개선하는 데 필수적인 방법론입니다.

## 수동 평가의 한계점

- 🕒 수백 개의 QA 샘플을 수동으로 생성하는 것은 **시간과 노동력이 많이 소요**
- ⚠️ 인간이 만든 질문은 철저한 평가에 필요한 **복잡성 수준에 도달하기 어려움**
- 📉 수동 평가는 **품질 일관성**을 유지하기 어려움

## 오프라인 평가의 이점

- ⚡ **효율성 향상**  
개발자의 시간을 최대 **90%**까지 단축
- 📊 **다양한 시나리오**  
복잡하고 다양한 테스트 케이스를 효율적으로 생성
- 🔍 **정밀한 분석**  
LLM 생성 데이터의 품질을 세밀하게 측정하고 분석



# 합성 데이터(Synthetic data) 생성의 필요성과 장점

## ! 수동 QA 샘플 생성의 한계



### 시간 소비

수백 개의 QA(질문-문맥-응답) 샘플을 수동으로 생성하는 과정은  
시간과 노동력이 많이 소요



### 복잡성 부족

인간이 만든 질문은 철저한 평가에 필요한 복잡성 수준에 도달하기  
어려움



### 품질 일관성

수동 평가는 평가 품질에 영향을 미칠 수 있는 일관성 부족 문제를  
갖고 있음

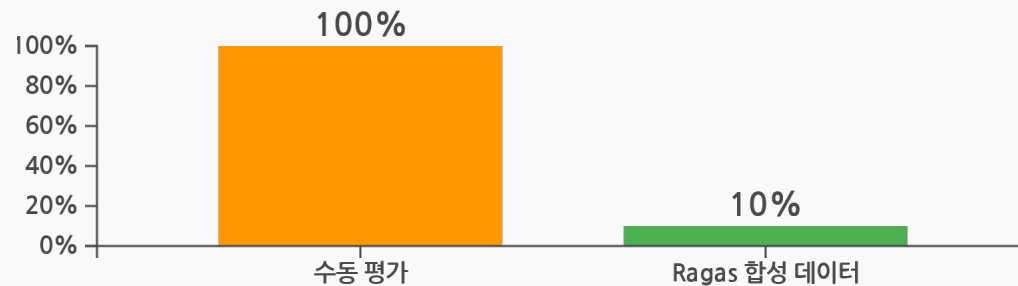
## 🚀 Ragas 합성 데이터의 장점



### 개발 시간 단축

Ragas를 활용한 합성 데이터 생성은 데이터 집계 프로세스에서  
개발자의 시간을 최대 90%까지 단축

### 개발 시간 단축 효과



LLM, 시뮬레이션 등을 통해 인공적으로 생성된 데이터를 합성 데이터(Synthetic data) 라고 합니다.

# 환경 설정 및 문서 전처리



## 라이브러리 설치

필요한 라이브러리들을 설치합니다:

```
!pip install ragas==0.3.7 datasets -U
!pip install langchain langchain-openai langchain-community
      langchain-text-splitters
!pip install langchain-openai>=0.2.0
!pip install faiss-cpu tiktoken python-dotenv
!pip install GitPython
```



## API 키 설정

환경 변수로 API 키를 설정합니다:

```
import os
from google.colab import userdata
os.environ["LANGSMITH_TRACING"] = "true"
os.environ["LANGSMITH_ENDPOINT"] =
"https://api.smith.langchain.com"
os.environ["LANGSMITH_API_KEY"] = userdata.get("LANGSMITH_API_KEY")
os.environ["LANGSMITH_PROJECT"] = "프로젝트명"
os.environ["OPEN_API_KEY"] = userdata.get("OPENAI_API_KEY")
```



## 문서 로드 및 전처리

평가 대상 문서를 로드하고 전처리합니다:

```
import pprint
from langchain_community.document_loaders import GitLoader
document_loader = GitLoader(
    clone_url="https://github.com/langchain-ai/langchain",
    repo_path="./langchain",
    branch="master",
    file_filter=lambda file_path: file_path.endswith(".md"),
)
documents = document_loader.load()
print(len(documents));
```



## 메타데이터 설정

filename 메타데이터를 설정하여 문서 식별:

```
for document in documents:
    document.metadata['filename'] =
document.metadata['source']
```

이 **filename** Ragas가 사용하는 메타데이터로 테스트 데이터셋 생성 과정에서 동일한 문서에 속한 청크를 식별하는 데 사용됩니다.

# TestsetGenerator를 이용한 합성 테스트 데이터셋 생성

## TestsetGenerator 개요

Ragas의 **TestsetGenerator**를 사용하면 RAG 시스템을 위한 합성 테스트 데이터셋을 생성할 수 있습니다. 이 데이터셋은 질문, 답변, 컨텍스트 등으로 구성됩니다.

### 생성된 데이터셋 구성 요소

- ✓ **user\_input** : 사용자 질문
- ✓ **reference\_contexts** : 모델이 답변을 생성할 때 참고해야 하는 컨텍스트
- ✓ **reference** : 정답/참조 답변
- ✓ **synthesizer\_name**: 질문 유형 분포

## 코드 예시

```
# TestsetGenerator 초기화
generator = TestsetGenerator(
    llm=generator_llm,
    embedding_model=generator_embeddings,
)

# 질문 유형별 분포 결정
query_distribution = {
    "single_hop_specific_query_synthesizer": 0.5, # 단일 문서를 기반으로 한 질문 50%
    "multi_hop_specific_query_synthesizer": 0.25, # 여러 문서 필요, 구체적, 특정 사실
    "multi_hop_abstract_query_synthesizer": 0.25 # 여러 문서 필요, 추상적, 개념/원리
}

# 테스트셋 생성
testset = generator.generate_with_langchain_docs(
    documents,
    testset_size=6,          # 6개의 질문 생성
    query_distribution=query_distribution
)

# 생성된 테스트셋을 pandas DataFrame으로 변환
test_df = testset.to_pandas()
```

# LangSmith 데이터셋 저장 및 관리

오프라인 평가에 사용할 데이터셋은 저장 관리가 중요합니다.

## 데이터셋 생성 및 업로드

```
from langsmith import Client

client = Client()
DATASET_NAME = "데이터셋 이름"

# 기존 데이터셋 확인
existing_dataset = None
for dataset in client.list_datasets(dataset_name=DATASET_NAME):
    if dataset.name == DATASET_NAME:
        existing_dataset = dataset
        break

# 새 데이터셋 생성 (없을 때만)
if not existing_dataset:
    dataset = existing_dataset
else:
    dataset = client.create_dataset(
        dataset_name=DATASET_NAME,
        description="데이터셋 설명"
    )
```

## 데이터셋에 Example 추가

DataFrame의 각 행을 LangSmith 예제(Example)로 변환하여 데이터셋에 추가합니다:

```
from ragas.testset.synthesizers.testset_schema import TestsetSample

for sample in testset.samples:
    inner = sample.eval_sample
    question = inner.user_input
```

## 데이터셋 관리 기능



### 버전 관리

예제(example)를 추가, 업데이트 또는 삭제할 때마다 새로운 버전이 생성되어 시간 경과에 따른 변경 사항을 추적할 수 있습니다. 특정 버전에 태그를 지정하여 중요한 이정표를 표시할 수 있습니다.



### 필터링 및 분할

특정 기준에 따라 데이터셋의 하위 집합을 가져와 평가에 활용할 수 있습니다. `list_examples` 메서드를 사용합니다.



### 공유 및 내보내기

데이터셋을 공개적으로 공유하거나 팀 내에서 협업할 수 있습니다. LangSmith UI에서 CSV, JSONL 또는 OpenAI의 미세 조정 형식으로 내보낼 수 있습니다.

# Ragas 주요 평가 메트릭

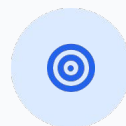
RAG(Retrieval-Augmented Generation) 시스템의 성능을 평가하기 위해 Ragas는 다양한 메트릭을 제공합니다. 메트릭들은 LLM(Large Language Model)을 평가자로 활용하여 생성된 답변의 품질과 검색된 컨텍스트의 관련성을 다각도로 측정합니다.



## Faithfulness

모델의 답변이 Retrieval 문서(Context)에 기반하여 사실 그대로 반영하고 있는가?

- 환각(factual hallucinations)을 방지하는 데 중점
- 생성된 답변의 각 주장이 컨텍스트에서 추론 가능한지 검증



## Answer Relevancy

질문에 대한 생성된 답변의 적합성 및 관련성을 평가합니다.

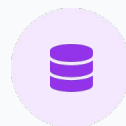
- 답변이 질문에 직접적으로 관련 있는지 측정
- 의도된 정보 전달의 효과를 평가



## Context Precision

검색된 컨텍스트가 질문 및 정답과 얼마나 관련성이 높은지 평가합니다.

- 검색된 컨텍스트의 질을 측정
- 정답에 필요한 정보만을 포함하도록 유도



## Context Recall

질문에 대한 답변에 필요한 모든 관련 정보가 검색되었는지 평가합니다.

- 검색된 컨텍스트가 충분한지 측정
- 답변에 필요한 모든 정보를 포함하고 있는지 평가

# 검색(Retriever) 관련 평가 지표

RAG 시스템에서 검색 성능은 외부 지식 기반에서 관련 정보를 얼마나 효과적으로 추출하는지를 평가합니다.

Ragas는 검색 관련 주요 지표를 통해 시스템의 검색 구성 요소를 평가할 수 있도록 합니다.

## Context Precision

✓ **정의:** 검색된 문서가 질문과 관련 있는가?

💡 **평가 목적:** 검색기(Retriever)가 반환한 문서들이 사용자 질의에 얼마나 적절하게 관련되어 있는지를 확인합니다.

낮은 점수: 검색된 문서들 중 관련성이 낮은 문서가 많음을 의미하며,

! 검색기의 재정렬(reordering) 기능이나 검색 전략을 개선해야 할 가능성이 있습니다.

## 예시

**질문:** 아인슈타인의 상대성 이론은 무엇인가?

검색된 문맥(3개)

1. 아인슈타인의 특수 상대성 이론 설명 → 관련 있음
2. 아인슈타인의 생애에 대한 이야기 → 반쯤 관련 (정답에 직접 필요 X)
3. 뉴턴의 만유인력 설명 → 관련 없음

-> 필요한 문맥: 1개

-> 검색된 문맥 총 3개

-> **Context Precision** =  $1/3 \approx 0.33$

검색된 문맥 중 대부분은 정답 구성에 필요하지 않은 정보이므로 Precision이 낮음.

# 검색(Retriever) 관련 평가 지표

RAG 시스템에서 검색 성능은 외부 지식 기반에서 관련 정보를 얼마나 효과적으로 추출하는지를 평가합니다.

Ragas는 검색 관련 주요 지표를 통해 시스템의 검색 구성 요소를 평가할 수 있도록 합니다.

## Context Recall

✓ **정의:** Ground Truth 정답을 만들기 위해 필요한 정보가 검색된 Context 문서에 모두 포함되어 있는가?

💡 **평가 목적:** 검색된 문서들이 정답에 필요한 모든 정보를 제공하는지, 또는 중요한 정보가 누락되지 않았는지를 평가합니다.(정보 누락 여부)

❗ **낮은 점수:** 외부 지식 기반에 포함된 중요한 정보를 누락했음을 의미하며, 검색기의 검색 범위나 문서 집합(document set)을 확장해야 할 가능성이 있습니다.

## 예시

**질문:** 광합성 과정은 어떻게 이루어지는가?”

정답에 실제로 필요한 문맥(ground truth)

광합성 설명에 필요한 문헌 2개:

1. 광합성의 정의
  2. 광합성의 화학 반응 과정
- > 실제 필요한 문맥 = 총 2개

모델이 검색해서 가져온 문맥

1. 광합성의 정의 -> 필요한 문맥
  2. 식물의 잎 구조 설명 -> 관련 있지만 정답 핵심에 직접 필요 X
  3. 광합성의 역사적 발견 과정 -> 필요 없음
- > 실제 필요한 문맥 2개 중 모델이 가져온 것은 1개만

**Context Recall =  $1 / 2 = 0.5$**

➡ 즉, 모델은 “정답을 구성하는 데 필요한 정보의 절반만 가져왔다”는 의미.

# 생성(Generator) 관련 평가 지표

RAG 시스템이 생성한 답변의 품질을 평가하기 위해 검색된 문맥의 충실도와 질문에 대한 답변의 관련성을 측정



## Faithfulness (충실도)

### 정의

생성된 답변이 검색된 문맥에 포함되지 않은 잘못된 정보를 포함하고 있는지 여부를 측정

### 평가 목적

검색된 정보를 얼마나 충실하게 반영했는지를 평가



점수가 낮게 나오는 경우

LLM이 생성하는 답변이 검색된 문맥에 포함되지 않은 잘못된 정보를 포함하고 있을 가능성이 큼 (Hallucination 발생)



## 예시

### 문서(Context):

“광합성 과정에서 식물은 물(H<sub>2</sub>O)을 분해하여 산소(O<sub>2</sub>)를 방출한다.”

### 모델 답변:

“광합성에서 식물은 물을 분해하고 산소를 방출하며, 동시에 포도당을 합성한다.”

### 모델 답변 주장 요소:

1. 물 분해 ✓
2. 산소 방출 ✓
3. 포도당 합성 ✗ (문서에 근거 없음)

Faithfulness 계산:  $2 / 3 \approx 0.67$

즉, 모델 답변 중 약 67%가 문서 기반으로 사실적이라는 의미입니다.



# 생성(Generator) 관련 평가 지표

RAG 시스템이 생성한 답변의 품질을 평가하기 위해 검색된 문맥의 충실도와 질문에 대한 답변의 관련성을 측정



## Answer Relevancy (답변 관련성)

### 정의

정답이 사용자 질의에 얼마나 관련 있는지를 측정

### 평가 목적

정답의 관련성과 질문에 대한 직접적인 관련성을 평가



점수가 낮게 나오는 경우

생성된 답변이 사용자 질문에 답변으로서 적절하지 않거나,  
질문과 관련 없는 내용을 포함하고 있을 가능성이 큼



## 예시

**질문:** 광합성 과정에서 산소는 어떻게 생성되나요?

Ground truth answer 핵심 요소

1. 물이 분해됨.
2. 산소 방출
3. 전자와 에너지 생성

-> 핵심 요소 총 3개

**모델 답변:** 광합성에서 식물은 엽록체 안에서 빛 에너지를 사용하여 물( $H_2O$ )을 분해하고 산소( $O_2$ )를 방출합니다.

포함된 관련 요소:

1. 물이 분해됨.
2. 산소 방출
3. 전자와 에너지 생성 x

-> Answer Relevance =  $2 / 3 \approx 0.67$

질문 의도는 어느 정도 충족하지만 일부 중요한 정보는 누락됨

# 결론 및 기대 효과

## 통합된 오프라인 평가 파이프라인의 효과



### 시간 및 노동력 절감

수동 데이터 구축의 **최대 90% 절감** - 효율적인 테스트셋 확보



### 투명하고 재현 가능한 평가

LangSmith를 통한 평가 프로세스의 **투명성과 재현성 확보**



### 정밀한 분석 capability

Ragas 메트릭을 통한 **LLM 생성 데이터의 품질 세부 분석**

## RAG 시스템 품질 향상 기여도



### 신뢰성 향상

검증된 평가 메트릭을 통해 RAG 시스템의 **신뢰성 높임**



### 지속적 성능 개선

체계적인 평가 기반 **반복적 개선 가능**



### LLM 기반 애플리케이션 품질 향상

최종적으로 **LLM 기반 애플리케이션의 전반적인 품질 향상**으로 이어짐

LangSmith와 Ragas의 통합은 RAG 시스템 개발 및 최적화를 위한 체계적인 접근 방식을 제공합니다.